AquaRun

Automated Community Water Refill, Dispatch & Bottle Ledger System

System Documentation
Version 2.0 · October 2026
University of Caloocan City

1. System Summary & Purpose

AquaRun is a lightweight, responsive web application engineered specifically for community water refilling stations operating within urban Philippine settings (specifically Caloocan City). The system automates and integrates five essential refilling station operations:

Online Refill Booking

Customer self-service ordering for 5-gallon container refills and bottle purchases with flexible slot scheduling.

Station Workflow

Order queue monitoring, slot window capacity balancing, and washing/refilling station status transitions.

Batch Dispatch

Barangay-clustered order assignment for riders, optimizing multi-stop neighborhood delivery routes.

Bottle & Cash Ledger

Exact gallon exchange tracking (delivered vs empties) and two-stage cash turnover reconciliation.

2. Complete Technology Stack

COMPONENT	TECHNOLOGY / LIBRARY	IMPLEMENTATION DETAILS & ARCHITECTURAL ROLE
Frontend Architecture	Vanilla JavaScript (ES Modules), HTML5, CSS3 Variables	Pure, zero-dependency client architecture. Modular ES6 imports (`/core` , `/services` , `/pages`) offering ultra-fast page load times (<0.5s) without bundle compilation or virtual DOM overhead.
Database & Storage	Supabase (PostgreSQL 15+ Cluster)	Relational storage with strict foreign keys, atomic transactions, row-level locks (`FOR UPDATE`), check constraints, and computed SQL views for balances and sales.
Authentication	Supabase Auth (GoTrue) + JWT	Stateless JSON Web Tokens with client-side session caching, encrypted credentials, automated email confirmation, and metadata-driven user profiling.
Access Control (RBAC)	PostgreSQL Row Level Security (RLS)	SQL security policies enforced natively at the database level. Ensures customer isolation and prevents unauthorized role impersonation.
Real-Time Synchronization	Supabase Realtime (WebSockets) + Web BroadcastChannel	PostgreSQL change-data-capture (CDC) pushes live updates to connected clients, while the HTML5 BroadcastChannel API synchronizes state across open browser tabs instantly.
Cloud Hosting & CI/CD	Vercel & Netlify Edge Infrastructure	Atomic Git deployments from `main` branch with global edge routing and HTTP cache headers to prevent stale assets.

3. Architectural Principles

Jamstack & Serverless

Static markup served from edge CDN paired with serverless PostgreSQL queries via client SDK—zero backend server maintenance.

ACID Data Guarantees

Inventory, bottle balances, and remittances are processed in atomic PL/pgSQL procedures to eliminate race conditions.

Mobile-First Responsive

Optimized for mobile touchscreens (rider delivery runsheet & customer checkout) with zero horizontal overflow.

4. APIs & Communication Protocols

1. Supabase Client SDK (@supabase/supabase-js)

Connects securely via public anon key to perform parameterized REST queries, schema filtering, pagination, and multi-table joins.

2. Database RPC (Remote Procedure Calls)

Calls stored SQL routines for transactional operations: `create_order()`, `mark_ready()`, `record_delivery()`, `staff_approve_remittance()`.

3. Browser BroadcastChannel API

Communicates on channel `aquarun_sync`. When staff or admin updates products, orders, or slots, all open tabs refresh live data instantly.

4. Native Device Telephony Protocols

Integrates `tel:+63...` and `sms:+63...` URIs on rider delivery cards for instant voice and SMS customer coordination without leaving the web app.

5. Role-Based Access Control (RBAC) Matrix

ROLE	WORKSPACE BASE	ALLOWED CAPABILITIES	DATA ACCESS BOUNDARIES
CUSTOMER	`/customer/*.html`	Browse catalog, save delivery addresses, submit refill bookings, track active orders in real time, view container balance ledger.	Can only read and modify records where `customer_id = auth.uid()`.
STATION STAFF	`/staff/*.html`	Confirm orders, adjust scheduled slots, fast-track refilling, batch assign orders to active riders, verify and accept cash remittances.	Operational access across all station orders and rider remittances; cannot modify administrative settings.
RIDER	`/rider/*.html`	View assigned delivery run sheet, toggle station availability, log delivered gallons and collected empties, submit cash remittance.	Can only read and update orders assigned directly to their rider account (`rider_id = auth.uid()`).
ADMIN	`/admin/*.html`	Executive analytics, 7-day/30-day hourly pattern charts, revenue reports, slot capacity controls, product catalog pricing, staff/rider accounts.	Full administrative privileges across all system tables, configurations, and user records.

6. Security & Data Integrity Safeguards

- **PostgreSQL Row Level Security (RLS):** Every database table has RLS enabled with distinct policies for `select`, `insert`, `update`, and `delete`.
- **Public Key Isolation:** Only the public Supabase Anon key is bundled in client scripts. The Service Role key is strictly prohibited, ensuring zero privilege escalation risks.
- **Atomic Ledger Triggers:** Bottle balances are never manually edited; they are computed dynamically via SQL view (`customer_bottle_balance`) from immutable ledger transactions.

7. End-to-End Operational Workflow

STEP 1 · ORDER PLACEMENT (CUSTOMER)

Customer Selects Refill Type & Delivery Window

Customer chooses water product (Round Gallon, Slim Gallon, or Bottle), specifies whether it is a standard 1:1 container swap (₱30-₱35) or a new gallon purchase (+container deposit), selects delivery address and preferred slot.

STEP 2 · QUEUE INGESTION & SLOT CONFIRMATION (STAFF)

Station Confirms & Validates Slot Capacity

The order appears on the Staff Orders queue in real time. Staff verifies queue capacity, confirms the scheduled window, or adjusts the delivery time slot if the window is fully booked.

STEP 3 · WATER PURIFICATION & PACKAGING (STAFF)

Washing, Refilling & Ready Status

Staff washes and sanitizes empty containers, refills them with purified drinking water via the station's reverse osmosis/UV equipment, seals the caps, and marks the order as "Ready" for delivery.

STEP 4 · SMART BATCH DISPATCH (STAFF → RIDER)

Clustering by Barangay & Route Assignment

Staff groups ready orders located in the same barangay or delivery corridor into an optimized batch run and assigns them to an active, available rider. The order transitions to "Out for Delivery".

STEP 5 · DOORSTEP FULFILLMENT & LEDGER RECORDING (RIDER)

Gallon Handover, Empty Collection & COD Payment

Rider delivers purified gallons to the doorstep, collects empty swapped bottles, and receives cash payment. On the mobile workbench, the rider records the exact count of delivered bottles and collected empties, creating an immutable ledger entry.

STEP 6 · SHIFT RECONCILIATION & REMITTANCE TURNOVER (RIDER → STAFF)

Cash Turnover & Ledger Settlement

At the end of the shift or delivery run, the rider returns to the station and submits their cash remittance report. Station staff counts the physical cash against the digital total in the Cash & Sales workbench and accepts the remittance with 1 click.

8. Specialized Business Rules

1:1 Empty Gallon Exchange Rule

If a customer returns fewer empty bottles than gallons ordered, the difference is logged into their container ledger. The customer's net balance of station bottles held increases automatically.

Dual Pricing Engine

Seamlessly handles two price points per product: Refill price (for customers providing empties) vs. Full Container Price (refill + brand new container purchase fee).

9. Relational Database Schema

TABLE / VIEW	TYPE	COLUMNS & RELATIONAL FUNCTIONALITY
`profiles`	Table	`id (PK, FK auth.users), full_name, phone, role, is_active, is_available, created_at` — Core user identities.
`addresses`	Table	`id, customer_id (FK), label, street, barangay, city, landmark, is_default` — Delivery destination geodata.
`products`	Table	`id, name, description, refill_price, new_container_price, is_active` — Refill product catalog.
`orders`	Table	`id, order_number, customer_id (FK), rider_id (FK), status, subtotal, service_charge, total, payment_status, remittance_status`
`order_items`	Table	`id, order_id (FK), product_id (FK), quantity, unit_price, is_new_container` — Itemized transaction details.
`bottle_ledger`	Table	`id, customer_id (FK), order_id (FK), bottles_delivered, empties_collected, recorded_by` — Container ledger log.
`time_slots`	Table	`id, label, start_time, end_time, max_orders, is_active` — Operating capacity and slot schedule.
`customer_bottle_balance`	View	Computes net balance: `Σ(bottles_delivered) - Σ(empties_collected)` per customer profile.
`daily_report`	View	Calculates daily paid orders, refill sales, service charge totals, and pending cash collections.

10. Key Innovations

Live Order Tracking Without AI Clutter

Real-time visual timeline showing confirmed, refilling, out for delivery, and completed stages without generic mockups.

Two-Stage Remittance Security

Riders submit digital remittance reports upon station return; staff must physically verify the cash before marking settled.

11. Production Environment & Hosting Specs

- **Live Production URL:** https://aquarun-puce.vercel.app
- **Database Infrastructure:** Supabase Cloud PostgreSQL Cluster (Asia-Southeast / Singapore region).
- **Edge Invalidation:** Configured with `vercel.json` cache-control rules ensuring zero client caching issues during deployments.
- **Repository & CI/CD:** GitHub repository (`Willy438/aquarun`) with automatic production webhooks.